

AI Compiler Performance Engineering

Auto-Loops Books — ch01

Contents

1 LLM Inference Performance Bottlenecks: From Symptoms to Root Causes	1
1.1 Prefill and Decode: Two Performance Regimes	1
1.2 The Decode Pain Point: Single-Token, Low-Batch Latency	3
1.3 Limits of Mainstream PyTorch and Hugging Face Inference	4
1.4 Quantifying Overheads: Launches, Memory, and Scheduling	5
1.5 Memory Bandwidth, Not Peak FLOPs	6
1.6 Multi-Hardware Pain Points: GPU, CPU, and NPU	7
1.7 Industry Benchmarks: Native Frameworks vs. Fused Kernels	8
1.8 Opening Case Study: YiRage Cross-Hardware Compilation Preview	9
1.9 Chapter Summary	14
Bibliography	15
Index	17

Chapter 1

LLM Inference Performance Bottlenecks: From Symptoms to Root Causes

Large language model (LLM) serving has matured into a systems discipline where the dominant cost is often not FLOPs but *how* weights, activations, and the key-value (KV) cache move through memory hierarchies and how many discrete GPU kernels the host must launch per token. In production, most teams hit the same contradiction: prefill benchmarks look excellent on datasheets, yet interactive decode still misses p99 latency SLOs after months of kernel tuning. The durable misconception is that buying faster tensor cores or enabling FlashAttention “fixes” decode—when measured kernels-per-token and bytes-per-token often barely move because the stack remains operator-fragmented and host-orchestrated (Vellaisamy *et al.*, 2026; Chen, 2026). This chapter establishes the performance vocabulary used throughout the book: we separate **prefill** from **decode**, quantify where latency hides in mainstream PyTorch and Hugging Face stacks, and explain why cross-hardware compilation is not an optional portability exercise but a prerequisite for industrial decode optimization.

1.1 Prefill and Decode: Two Performance Regimes

Autoregressive transformer inference divides naturally into two phases (Davies *et al.*, 2025). In **prefill**, the model processes the entire prompt (or a prompt chunk) in parallel across sequence positions. Matrix multiplications are large, arithmetic



Figure 1.1: Prefill processes prompt chunks in parallel; decode appends one token per step via KV cache.

intensity is relatively high, and modern accelerators can approach their peak tensor throughput. In **decode**, the model generates one new token per forward pass (per request, in the common batch-1 interactive setting), conditioned on all prior tokens through the KV cache.

These phases obey different roofline limits. Prefill is frequently *compute-bound* on high-end GPUs: the GEMMs and attention blocks are wide enough to amortize memory latency. Decode, by contrast, repeatedly reads the full parameter set and growing KV tensors while performing only a thin slice of matmul work per token. Treating both phases with one optimization playbook—larger batches, flash attention defaults, tensor-core-first schedules—is a primary reason decode SLOs remain elusive after prefill tuning succeeds. Multiple independent analyses converge on the same conclusion: interactive decode is typically *memory-bandwidth bound* on GPUs, CPUs, and many NPUs, not FLOP-bound (Davies *et al.*, 2025; Chen, 2026).

Production systems increasingly *disaggregate* prefill and decode into separate pools because optimizing for one phase can harm the other. For example, large prefill batches maximize throughput, while decode pools prioritize tail latency under batch-1 or small-batch arrivals. Recent large-model deployments provision substantially more decode capacity than prefill capacity because user-visible latency is dominated by per-token decode time once the prompt has been ingested (Davies *et al.*, 2025; Liu *et al.*, 2024). DeepSeek-V3’s production layout is illustrative: the minimum prefill deployment unit spans 4 nodes (32 GPUs), whereas the minimum decode unit spans 40 nodes (320 GPUs)—a $10\times$ capacity skew reflecting phase-specific hardware mapping (Liu *et al.*, 2024).

1.2 The Decode Pain Point: Single-Token, Low-Batch Latency

Interactive assistants, coding copilots, and edge voice interfaces share a harsh constraint: users measure quality in *time-to-next-token*, often at batch size one. At batch-1 decode, each layer performs narrow GEMMs (e.g., a single new query

Table 1.1: Qualitative comparison of prefill vs. decode on a single transformer layer.

	Prefill	Decode (batch-1)
Tokens processed per step	L (prompt length)	1
Typical arithmetic intensity	Higher	Lower
Dominant limiter	Compute / tensor cores	Memory traffic / orchestration
KV cache growth per step	Written in bulk	Read + append one step
Serving objective	Throughput (tok/s)	Latency (ms/token)

vector against frozen weights) and attention that must touch the entire cached history. The working set per token is small, but the *read footprint* is enormous: every weight matrix and every cached key/value vector must be visited again. This is why decode latency scales with model width and context length even when FLOPs per token look trivial on paper—the hardware spends its time streaming bytes, not executing MMA instructions (Davies *et al.*, 2025; Chen, 2026).

Low batch sizes also prevent amortization of fixed costs—kernel launch, framework dispatch, collective synchronization, and memory allocator bookkeeping—across many tokens. Vellaisamy *et al.* (2026) decompose host-visible latency into three mutually exclusive components per kernel invocation: framework translation (ΔFT), CUDA-library front-end time (ΔCT), and kernel launch-path floor (ΔKT). Their Host–Device Balance Index (HDBI) relates device-active time to orchestration time; $\text{HDBI} \rightarrow 0$ marks a host-bound regime.

On H200 at batch-1 and sequence length 512, Llama-3.2-1B issues 850 kernel launches during prefill but 8,437 launches across a 10-token decode window (≈ 844 launches per autoregressive step)—nearly $10\times$ more dispatch work per forward pass (Vellaisamy *et al.*, 2026). At decode configuration $\text{BS}=4/\text{SL}=2048$ ($m=10$ output tokens), dense Llama-3.2-1B averages 847.5 GPU kernel launches *per output token*; OLMoE-1B/7B averages 9,305 launches per token ($11\times$ denser) (Vellaisamy *et al.*, 2026). Measured null-kernel launch floor on H100 is $4.707\ \mu\text{s}$ (median $4.578\ \mu\text{s}$); at 9,305 launches/token that launch-path term alone contributes $\approx 44\ \text{ms}$ per token before framework dispatch is counted (Vellaisamy *et al.*, 2026).

The engineering implication is blunt: shaving a few microseconds from a matmul inner loop matters less than eliminating redundant global-memory round trips, fusing layer fragments, and collapsing launch storms—especially when the product SLA is sub-50 ms per token.

1.3 Limits of Mainstream PyTorch and Hugging Face Inference

The conventional PyTorch + Hugging Face stack looks logically complete—every transformer sub-block maps to a familiar op—yet multi-hardware deployment exposes a fatal gap: the *schedule* is decided at runtime by a Python loop, not at compile time by residency and bandwidth constraints. A naïve **transformers** loop—one Python forward per token, eager PyTorch ops, separate kernels for layer norm, QKV projection, attention, MLP, residuals—mirrors the mathematical graph but not the memory hierarchy. Each operator materializes intermediates to global memory (GPU HBM or CPU DRAM), so a single decoder layer can trigger many read/write passes over tensors that could have stayed on chip given a different schedule.

Three structural problems recur:

1. **Operator fragmentation.** A decoder block decomposes into 10–20+ discrete kernels in unfused stacks. Each boundary exports data to HBM and re-imports it for the next op.
2. **Host orchestration tax.** Python dispatch, CUDA driver launch, and framework bookkeeping run on the CPU while the accelerator idles between micro-kernels (Vellaisamy *et al.*, 2026).
3. **KV cache management overhead.** Without paged or fused cache layouts, attention reads scatter through memory, wasting bandwidth on indexing and padding (Kwon *et al.*, 2023; Dao *et al.*, 2022).

Serving frameworks such as vLLM address *capacity* and *KV fragmentation* via PagedAttention and continuous batching (Kwon *et al.*, 2023), but they do not by themselves remove per-layer operator boundaries inside the forward pass. FlashAttention reduces HBM traffic during prefill by IO-aware tiling (Dao *et al.*, 2022); Flash-Decoding adds a parallelization dimension along KV sequence length so batch-1 decode can still saturate SMs when context is long (reported up to $8\times$ end-to-end speedup at very long sequences) (Dao *et al.*, 2023). These are necessary optimizations, yet a full decoder step still contains MLPs, norms, rotary embeddings, and sampling logic that remain separable kernels unless a **MegaKernel** or compiler-generated fused schedule reunifies them.

On CPUs the same fragmentation shows up as LLC misses rather than HBM stalls: each unfused boundary evicts cache lines that a fused decoder block could have kept resident across norm–matmul–activation chains. On XDNA-class NPUs,

Table 1.2: Verified kernel fragmentation on H100 decode (BS=4, SL=2048, $m=10$ tokens), from TaxBreak (Vellaisamy *et al.*, 2026).

Model	Kernels/token	Total launches	GPU util. (%)
Llama-3.2-1B	847.5	8,475	58.9
Llama-3.2-3B	1,536.9	15,369	67.6
OLMoE-1B/7B	9,305.3	93,053	15.5
Qwen1.5-MoE-A2.7B	6,695.1	66,951	27.7

per-op dispatch is worse than wasteful—it violates static dataflow contracts and forces fallbacks to scalar host loops (AMD, 2024b). Industrial deployment therefore cannot copy a single “GPU recipe” into production; the framework graph must be re-planned per memory hierarchy, which is exactly the job YiRage takes over in Part VI.

1.4 Quantifying Overheads: Launches, Memory, and Scheduling

Recent microbenchmark studies decompose end-to-end decode latency into compute, memory, and *orchestration* components. TaxBreak reports that for Llama-3.2-1B-class models on H100, decode reduces orchestration time relative to prefill in absolute terms, but because decode issues roughly $10\times$ more launches per step, per-dispatch savings compound more aggressively in decode (Vellaisamy *et al.*, 2026). For MoE models, decode latency can grow to multiple seconds per token at batch-1 not because GEMMs are huge, but because dispatch dominates: thousands of kernels per token with a per-launch floor on the order of microseconds yields tens of milliseconds of pure launch-path cost before framework overhead (Vellaisamy *et al.*, 2026).

CUDA Graphs record a static kernel DAG once and replay it with a single submission, amortizing CPU launch and framework dispatch (NVIDIA, 2024; Chen, 2026). Chen (2026) ran a controlled CUDA Graphs A/B on Qwen-2.5-7B at batch-1, context 2048, across $N=10$ independent sessions: H100 speedup is $1.259\times$ (95% bootstrap CI [1.253, 1.267]), removing ≈ 3.05 ms of per-step host overhead ($\approx 20.6\%$ of the eager 14.83 ms median). On L4 at the same cell, Graphs yield only $1.028\times$ because the GPU already achieves $R_{\text{floor}}=0.81$ of its analytic HBM bandwidth floor—the device waits on memory, not launches (Chen, 2026).

Example 1.4.1. For Qwen-2.5-7B on H100, a single decode step issues ≈ 283



Figure 1.2: Decode sits on the memory-bandwidth slope of the roofline at low arithmetic intensity.

kernel launches (≈ 10 per layer $\times$ 28 layers plus embedding/LM-head) (Chen, 2026). At batch-1 interactive decode, per-layer GPU compute can be $\approx 10 \mu\text{s}$ while launch gaps are $\approx 30 \mu\text{s}$ —launch dominates per block (Chen, 2026). Collapsing those boundaries via MegaKernel fusion or graph capture can exceed the benefit of micro-optimizing individual GEMMs.

1.5 Memory Bandwidth, Not Peak FLOPs

The roofline model makes the decode story precise. Arithmetic intensity (FLOPs per byte moved) for batch-1 decode is low: one token’s activations participate in thin matrix–vector-like operations while weights and KV cache supply the bulk of bytes. Accelerators therefore sit on the memory-bandwidth sloped region of the roofline rather than the flat compute cap (Davies *et al.*, 2025; Chen, 2026). The LIMINAL limit study further reports that at low batch, tensor-core utilization can be $\leq 1\%$ on both DRAM- and SRAM-based accelerator designs when compute is reasonably provisioned—decode simply does not present enough arithmetic intensity to engage peak FLOPs (Davies *et al.*, 2025).

Chen (2026) quantify “achieved fraction of peak bandwidth” via $R_{\text{floor}} = t_{\text{floor}}/t_{\text{obs}}$ where $t_{\text{floor}} = (W+K)/B_{\text{peak}}$. For Qwen-2.5-7B at $\text{ctx}=2048$, batch-1: L4 reaches $R_{\text{floor}}=0.81$ (near the HBM roofline), while H100 reaches only $R_{\text{floor}}=0.27$ —faster silicon realizes a smaller fraction of theoretical memory throughput because fixed per-step launch overhead does not scale with bandwidth (Chen, 2026).

This explains why peak TFLOPs and peak HBM bandwidth are misleading single-number KPIs for chat latency: on H100-class GPUs the gap is often orchestration-limited; on L4-class GPUs it is bandwidth-limited. It also explains why decode often *should not* default to tensor-core-oriented kernels: the bottleneck is feeding data, not lacking MMA throughput—a theme we develop for Hopper/Blackwell in later chapters and for fused MegaKernels in Part V.

Quantization reduces bytes moved and can shift decode toward compute-bound regimes on some GPUs, but only when the runtime actually realizes memory savings. On L4 with Qwen-2.5-7B at ctx=2048, bf16 decode median is 62.32 ms while bitsandbytes nf4 is 59.36 ms and AutoAWQ+Marlin is 45.24 ms—far less than the $4\times$ traffic reduction would predict. GPTQ+ExLlamaV2 (Ada-tuned int4 GEMMs) reaches 17.36 ms, a $3.59\times$ speedup over bf16; an L4 with ExLlamaV2 (17.36 ms/step) can approach an H100 with CUDA Graphs (11.78 ms/step) despite $\approx 11\times$ lower peak HBM bandwidth (Chen, 2026).

1.6 Multi-Hardware Pain Points: GPU, CPU, and NPU

The decode bottleneck *shape* is universal; the *remedy* is not.

NVIDIA and AMD GPUs. Warp occupancy, shared memory tiling, coalesced global loads, and tensor-core vs scalar-core selection dominate. Launch overhead and CUDA Graph capture matter most when compute kernels are already short—exactly the decode regime on H100 (Chen, 2026; Vellaisamy *et al.*, 2026). AMD HIP stacks mirror many CUDA concerns but differ in wavefront width, LDS sizing, and ROCm graph support.

x86 and ARM CPUs. Decode is still memory-bound but the hierarchy is cache- and NUMA-shaped rather than HBM-shaped. SIMD width (AVX-512/NEON), core pinning, and KV layout for cache locality replace shared-memory tiling. Operator fusion reduces DRAM round trips; excessive parallelism can thrash LLC without NUMA-aware allocation. A common pitfall is porting GPU-oriented fusion heuristics verbatim: medium-grain CUDA fusion that helps H100 can *inflate* CPU decode latency when it destroys cache blocking or forces cross-socket KV traffic on dual-socket servers. For CPU decode, the first lever is often KV tensor layout (contiguous head dimensions, page size aligned to cache lines) and coarse layer fusion that keeps QKV+MLP fragments in L3 for the duration of a token step—not micro-kernels tuned for warp occupancy.

Edge NPUs (XDNA/AIE and similar). Spatial dataflow arrays (AMD XDNA) emphasize static tile assignment, on-chip buffer residency, and DMA

Table 1.3: Decode optimization priorities by hardware class.

Hardware	First-order decode levers
NVIDIA GPU	Fusion, graphs, KV bandwidth, shared mem tiling
AMD GPU	HIP fusion, LDS reuse, launch reduction
CPU	Cache blocking, SIMD, NUMA, coarse fusion
Edge NPU	Static ADF graphs, DMA ping-pong, full-layer fusion

pipelines with minimal dynamic scheduling (AMD, 2024b,a). Decode favors fully static graphs, large fused regions, and pre-planned memory reuse; dynamic shapes and per-op Python dispatch are architectural mismatches. The compiler must allocate buffers, routes, and DMA descriptors at compile time rather than relying on a runtime kernel launcher. Where CUDA permits flexible grid launches and runtime shape guards, AIE tiles require fixed ADF graphs: if the compiler cannot prove tensor lifetimes fit on-chip SRAM, the schedule fails bring-up rather than degrading gracefully to DRAM-heavy eager paths. That asymmetry is why NPU decode optimization ranks *static fusion* and *DMA ping-pong* above attention-kernel swaps—the binding constraint is residency proof, not FLOP count.

1.7 Industry Benchmarks: Native Frameworks vs. Fused Kernels

Industry leaderboards rarely tell the decode story teams actually ship against. Published prefill throughput (tokens/s on long prompts) and single-kernel attention speedups are real contributions, but they answer a different question than batch-1 ms/token under continuous batching with growing KV state. The technique is sound in theory—FlashAttention, PagedAttention, CUDA Graphs each remove a documented bottleneck—yet in live serving the stack often regresses because remaining layer fragments still export activations to HBM and the host still pays thousands of launches per token on MoE stacks (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Benchmark literacy therefore means tracking normalized diagnostics—kernels/token, bytes/token, R_{floor} , HDBI—on *identical* model revisions, not comparing marketing peaks across mismatched batch policies.

Published baselines establish the performance gap that motivates this book’s MegaKernel and compiler stack. Flash-Decoding reports up to $8\times$ faster generation at very long contexts by parallelizing KV loads; attention-only speedups

[Placeholder] Run Appendix B harness before filling bars

Figure 1.3: Pending: YiRage/MegaKernel vs. PyTorch eager speedup (Appendix B harness).

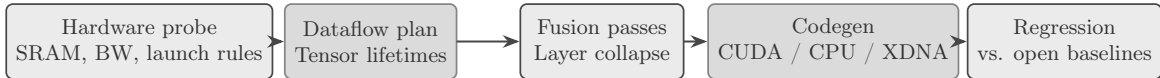


Figure 1.4: YiRage decode compilation pipeline (preview).

can reach $50\times$ versus unfused FlashAttention at batch-1 because standard attention parallelizes poorly when query length is 1 (Dao *et al.*, 2023, 2022). vLLM demonstrates that serving-system optimizations (paged KV, continuous batching) improve throughput under concurrent requests but leave per-kernel efficiency inside the transformer block as a separate axis (Kwon *et al.*, 2023).

TaxBreak and related studies quantify that framework-level improvements cannot fully absorb launch inflation when models fragment into thousands of micro-kernels per token—especially MoE architectures (Vellaisamy *et al.*, 2026). The gap we target in later chapters is the additional uplift from *whole-decoder fusion*: replacing on the order of ten to twenty discrete kernels with one dataflow-planned MegaKernel that keeps activations in registers and shared memory across QKV, attention, MLP, and residuals. Exact speedups are model-, hardware-, and quantization-dependent; this book reports measurements through a unified benchmark harness (Appendix B) rather than anecdotal single-point claims.

1.8 Opening Case Study: YiRage Cross-Hardware Compilation Preview

YiRage is the book’s reference **hardware-aware** LLM compiler: it ingests high-level decoder graphs, models chip constraints (memory tiers, launch granularity, SIMD/SIMT width), and emits fused schedules for CUDA, CPU, and XDNA-class targets. This chapter’s role is to state the *problem* YiRage solves; subsequent parts develop the hardware abstractions, manual MegaKernel craft (Part V), and automated pass pipelines (Part VI).

A preview of the YiRage workflow for decode:

1. **Hardware probe.** Read bandwidth caps, on-chip SRAM sizes, and static-

scheduling rules for the target.

2. **Global dataflow plan.** Assign tensor lifetimes to register, shared/LDS, or global tiers before choosing individual ops.
3. **Fusion and pipeline passes.** Collapse decoder layers; insert async copy pipelines (TMA on Hopper, DMA on AIE) where hardware supports them (NVIDIA, 2022; AMD, 2024a).
4. **Backend code generation.** Emit a single kernel or captured graph per layer group; tune tile sizes per architecture.
5. **Regression against open baselines.** Compare to PyTorch eager, Flash-Decoding, and framework servers on identical models (Pythia-2.8B, Qwen2.5 family) across GPU, CPU, and NPU targets.

The preview case establishes performance baselines and documents compiler constraints discovered during bring-up—not to claim a single universal speedup factor, but to show that *hardware-aware dataflow compilation* closes the gap between hand-tuned CUDA and production-ready multi-target deployment.

Cross-hardware bring-up surfaces constraints that single-GPU books omit. A fusion pass that succeeds on CUDA may fail XDNA buffer allocation; a CPU backend may require different tile shapes to keep AVX units fed without blowing L2. YiRage encodes these outcomes as *hard failures* during compilation (illegal residency, launch granularity mismatch) rather than silent slowdowns at runtime—matching how industrial teams gate releases. This preview chapter therefore ends with requirements R1–R5 (§1.8.5) that every later pass must satisfy with measured regression evidence, not anecdotal speedups on one card.

1.8.1 Diagnostic Workflow: TaxBreak and Memory-Floor Studies

Before optimizing kernels, industrial teams must answer: is this workload host-bound or device-bound, and *which host layer* dominates? TaxBreak answers the first decomposition: reduce $T_{\text{Orchestration}}$ via `torch.compile` when $\sum \Delta FT + \sum \Delta CT$ dominates; fuse kernels or adopt CUDA Graphs when $N \cdot T_{\text{sys}}^{\text{floor}}$ dominates; tune driver/runtime when per-kernel ΔKT_{fw} residuals are large (Vellaisamy *et al.*, 2026). For MoE decode, TaxBreak shows persistently low HDBI even at BS=16: routing fragments execution into 8–11× more launches per token than dense models of comparable activated parameter count, and faster H200 hosts reduce orchestration 10–29% versus H100—CPU single-thread performance is a first-order knob for host-bound decode (Vellaisamy *et al.*, 2026).

The memory-floor study complements TaxBreak on the device side: when R_{floor} is high (L4, ≈ 0.8), optimize bytes moved (quantization kernels that actually stream int4 weights, KV layout). When R_{floor} is low (H100, ≈ 0.27), optimize launches (graphs, fusion) before chasing attention-kernel swaps—on H100 batch-1 decode, explicit FlashAttention-2 eager paths can be *slower* than PyTorch SDPA because attention is not the binding constraint (Chen, 2026; Vellaisamy *et al.*, 2026). This book’s YiRage passes encode both diagnostics as regression gates (Appendix F).

1.8.2 Roofline Intuition for Decoder Layers

The roofline model plots attainable FLOP/s as $\min(\text{peak FLOP/s}, \text{bandwidth} \times \text{AI})$, where arithmetic intensity (AI) is FLOPs per byte transferred from DRAM (Davies *et al.*, 2025). For a decoder layer at batch-1, AI is roughly

$$\text{AI} \approx \frac{2N_{\text{params}} + 2N_{\text{KV}}}{N_{\text{params}} + N_{\text{KV}}} \quad (1.1)$$

in the crudest weight+KV read model (ignoring activations and writebacks), where N_{KV} grows linearly with context length. As context grows, AI rises slowly but usually remains on the bandwidth-limited slope for practical LLM sizes on GPUs. This is why Flash-Decoding and fused attention kernels fight for *bytes* and *launch count*, not raw MMA throughput (Dao *et al.*, 2022).

1.8.3 Why Disaggregated Serving Changes the Optimization Target

When prefill and decode run on separate fleets, decode nodes see a steadier stream of single-token steps with hot weights resident on device. That shifts engineering effort toward (i) per-step latency, (ii) KV cache locality, and (iii) kernel persistence (CUDA Graphs, MegaKernels) rather than maximizing prompt throughput (Davies *et al.*, 2025; Kwon *et al.*, 2023). Disaggregation also exposes *network* and *serialization* costs if KV tensors move between pools—another reason compile-time dataflow planning must account for where tensors live across steps.

1.8.4 Tensor Cores and Decode: A Common Misconfiguration

Tensor cores excel when GEMMs present sufficiently large $M \times N \times K$ tiles. Decode batch-1 often reduces effective M to one (or a tiny batch), shrinking MMA utilization and increasing setup overhead (SemiAnalysis, 2024). Hopper’s Tensor Memory Accelerator (TMA) shifts address-generation and bulk copy work off

scalar threads, but TMA still favors transfers large enough to amortize descriptor setup—small KV slices may remain latency-bound on regular loads (NVIDIA, 2022; SemiAnalysis, 2024). The book’s MegaKernel strategy therefore treats tensor cores as *optional epilogues* inside a dataflow schedule that first minimizes bytes and synchronization, then maps surviving GEMMs to MMA when dimensions justify it.

1.8.5 From Symptoms to Design Requirements

The pain points in this chapter translate into concrete compiler and kernel requirements used in Parts IV–VII:

- **R1 — Minimize global round trips.** Keep decoder activations in registers or shared memory across matmul, norm, activation, and attention phases.
- **R2 — Static schedules where possible.** Prefer compile-time tile assignment, especially on NPUs with spatial dataflow (AMD, 2024b).
- **R3 — Hardware-shaped fusion granularity.** GPUs tolerate medium fusion; CPUs need cache-aware blocking; NPUs require layer-scale or model-scale static graphs.
- **R4 — Launch amortization.** Use graphs, persistent kernels, or single MegaKernels to remove host dispatch from the critical path (Vellaisamy *et al.*, 2026; Chen, 2026).
- **R5 — Measurable regression gates.** Every pass must show bytes/token and kernels/token improvements on a fixed benchmark matrix.

These requirements are the bridge to chapter 1’s successor chapter on dataflow-first kernel design.

1.8.6 Measurement Methodology Used in This Book

We report decode latency as **median ms/token** at batch-1 unless stated otherwise, with warmup and fixed context lengths (512, 2048, 8192). Throughput experiments use continuous batching where noted. Every claim ties to a reproducible configuration: model revision, precision, framework version, driver, and compiler commit. Placeholder rows in Appendix B mark experiments pending hardware access; we never interpolate fake speedups. Cross-hardware tables normalize by *bytes moved per token* and *kernels launched per token* so readers can compare mechanisms, not marketing peaks (Vellaisamy *et al.*, 2026; Chen, 2026).

Table 1.4: Literature-backed decode metrics referenced in this chapter (conditions vary by row).

Metric	Value	Source / conditions
Prefill vs decode node ratio	$10\times$ decode	DeepSeek-V3: 32-GPU prefill unit vs 320-GPU decode unit (Liu <i>et al.</i> , 2024)
Dense kernels / decode step	≈ 844	Llama-3.2-1B, H200, BS1/SL512, vs 850 prefill (Vellaisamy <i>et al.</i> , 2026)
MoE kernels / output token	9,305	OLMoE-1B/7B, H100, BS4/SL2048, $m=10$ (Vellaisamy <i>et al.</i> , 2026)
Launch floor ΔKT	$4.707\ \mu\text{s}$	H100 null-kernel median (Vellaisamy <i>et al.</i> , 2026)
CUDA Graphs speedup	$1.259\times$	Qwen-2.5-7B, H100, ctx2048, $N=10$ sessions (Chen, 2026)
CUDA Graphs on L4	$1.028\times$	Same cell; bandwidth-bound null result (Chen, 2026)
R_{floor} H100 / L4	$0.27 / 0.81$	Qwen-2.5-7B, ctx2048, batch-1 (Chen, 2026)
Low-batch tensor util.	$\leq 1\%$	LIMINAL limit study, DRAM/SRAM xPU (Davies <i>et al.</i> , 2025)
Flash-Decoding speedup	up to $8\times$	Very long context, attention parallelized on KV (Dao <i>et al.</i> , 2023)

1.8.7 Verified Reference Numbers (Literature Snapshot)

Table 1.4 collects externally published measurements cited in this chapter. These are *not* YiRage internal benchmarks; they anchor the problem statement before we introduce book-specific experiments in later parts.

When migrating these lessons across hardware targets, compare normalized metrics—kernels per token, bytes per token, R_{floor} , HDBI—rather than absolute milliseconds, which entangle silicon generation, framework version, and batching policy.

1.8.8 Anti-Patterns That Waste Engineering Cycles

Three optimization anti-patterns recur in decode programs that miss SLOs despite heavy investment. **(1) Chasing peak FLOPs.** Teams profile GEMMs, tune tile

sizes, and declare victory while launch storms and HBM re-reads dominate ms/token (Vellaisamy *et al.*, 2026). **(2) Single-hardware tunnel vision.** A CUDA Graph capture that rescues H100 decode may be irrelevant on L4 (bandwidth-saturated) or illegal on static NPU targets (Chen, 2026; AMD, 2024b). **(3) Attention-only myopia.** Replacing SDPA with a flash variant when R_{floor} is already low on H100 can *slow* decode because attention was never the binding constraint (Chen, 2026). The corrective workflow is diagnostic-first: measure HDBI and R_{floor} , then select fusion depth, graph capture, quantization kernels, or compile-time dataflow replanning—in that priority order when orchestration dominates. Teams that skip this ordering often re-implement the same matmul tweak quarterly while p99 latency flatlines—because the root cause was never FLOPs, it was residency and dispatch. Industrial deployment cannot copy a single template; hardware class dictates whether graphs, fusion depth, or static NPU graphs come first on the critical path.

1.9 Chapter Summary

Decode-phase LLM inference is a systems problem disguised as a modeling problem. Prefill and decode obey different limits; interactive decode at low batch is dominated by memory traffic and orchestration overhead, not peak tensor FLOPs. Mainstream PyTorch and Hugging Face paths fragment layers into many kernels, amplify host launch costs, and under-utilize on-chip storage. GPUs, CPUs, and NPUs each demand different fusion granularities and compilation strategies—there is no portable “best kernel,” only hardware-shaped schedules. Industry kernels (FlashAttention, PagedAttention) and serving systems (vLLM) are necessary but insufficient for the extreme fusion pursued in this book. YiRage provides the automated, cross-hardware path from the manual MegaKernel insights developed next. This is the book’s recurring thesis: performance is a property of *data movement and orchestration under a specific memory hierarchy*, and compilers must own that schedule end-to-end.

Bibliography

- AMD (2024a). Ai engine technology. <https://www.amd.com/en/products/adaptive-socs-and-fpgas/technologies/ai-engine.html>.
- AMD (2024b). AMD XDNA architecture. <https://www.amd.com/en/technologies/xdna.html>.
- Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4.
- Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*.
- Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*.
- Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog.
- NVIDIA (2024). Cuda graphs. CUDA Programming Guide.
- SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter.

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*.

Index

Decode phase, 1

LLM inference, 1

Prefill phase, 1